

TECH

LIVE-CODING

JAVASCRIPT / ALGORITHMS TYPESCRIPT **REACT**

REACT

25 ЗАДАЧ 7 ДНЕЙ КОНСПЕКТ ЛЕКЦИЙ

ИСТОЧНИК [react.dev](#) / [common patterns](#)

20 ВОПРОСОВ 7 ДНЕЙ

ДЕНЬ 1 СРЕДНИЙ	USE-DEBOUNCE EVIOUS Q1 · Q2 · Q3	USE-THROTTLE	USE-PR	USE-DEBOUNCE EVIOUS	USE-THROTTLE	USE-PR
ДЕНЬ 2 СРЕДНИЙ	USE-FETCH RAGE Q4 · Q5 · Q6	USE-ASYNC	USE-LOCALSTO	USE-FETCH RAGE	USE-ASYNC	USE-LOCALSTO
ДЕНЬ 3 ТЯЖЁЛЫЙ	AUTOCOMPLETE Q7 · Q8	SEARCH WITH ABORT		AUTOCOMPLETE	SEARCH WITH ABORT	
ДЕНЬ 4 ТЯЖЁЛЫЙ	INFINITE SCROLL Q9 · Q10	VIRTUAL LIST		INFINITE SCROLL	VIRTUAL LIST	
ДЕНЬ 5 ТЯЖЁЛЫЙ	MODAL PORTAL DE CLICK Q11 · Q12 · Q13 · Q14	FOCUS TRAP	OUTSI	MODAL PORTAL DE CLICK	FOCUS TRAP	OUTSI
ДЕНЬ 6 ТЯЖЁЛЫЙ	FORM VALIDATION ACCORDION Q15 · Q16 · Q17 · Q18 · Q19	DROPDOWN	TABS	FORM VALIDATION ACCORDION	DROPDOWN	TABS
ДЕНЬ 7 ТЯЖЁЛЫЙ	DRAG SORT KANBAN Q20 · Q21 · Q22 · Q23 · Q24 · Q25	TIC-TAC-TOE	UNDO	DRAG SORT KANBAN	TIC-TAC-TOE	UNDO

РИТМ 30-60 МИН ТЕОРИЯ 15-30 МИН КОД В КОНСОЛИ 15 МИН ОТВЕТЫ ВСЛУХ
ДЕНЬ 7 — БЕЗ НОВОЙ ТЕОРИИ ТОЛЬКО ПРОГОН ПО 20 ВОПРОСАМ

ДЕНЬ 1 НАГРУЗКА СРЕДНИЙ 3 ВОПРОСА

USE-DEBOUNCE

USE-THROTTLE

USE-PREVIOUS

ЧТО УЧИМ В ЭТОТ ДЕНЬ

USE-DEBOUNCE USE-THROTTLE USE-PREVIOUS

ВОПРОСЫ ДНЯ

Q01 Напиши `useDebounce(value, delay)`

Q02 Напиши `useThrottle(value, interval)`

Q03 Напиши `usePrevious(value)`

Напиши `useDebounce(value, delay)`

ОПИСАНИЕ

Hook возвращающий debounced значение. `setTimeout` в `useEffect`, `clear` на `cleanup`.

КАК РАБОТАЕТ

```
01  useState(value).
02  useEffect: setTimeout setValue(v, delay).
03  return cleanup: clearTimeout.
04  deps [value, delay].
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01  Search input.
02  Filter inputs.
```

ПОДВОДНЫЕ КАМНИ

```
01  Без cleanup – старый timer не очищается.
02  initial value не debounce'нут.
```

МОГУТ ДОСПРОСИТЬ

```
01  Чем useDebounce от useThrottle отличается?
02  Как сделать leading-debounce?
```

ПОЛНАЯ СТАТЬЯ

<https://usehooks.com/usedebounce>

ВОПРОС Q02

Напиши `useThrottle(value, interval)`

ОПИСАНИЕ

Hook возвращающий throttled значение. lastUpdate ref + setTimeout.

КАК РАБОТАЕТ

- 01 useRef для lastUpdate timestamp.
- 02 useEffect: проверь elapsed.
- 03 Если elapsed $\geq$ interval – setValue.
- 04 Иначе setTimeout на остаток.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Scroll position.
- 02 Mouse position.

ПОДВОДНЫЕ КАМНИ

- 01 Без trailing – последний value пропущен.
- 02 Without ref – нельзя сохранить timestamp между renders.

МОГУТ ДОСПРОСИТЬ

- 01 Чем throttle от debounce отличается?
- 02 Что такое leading edge?

ПОЛНАЯ СТАТЬЯ

<https://github.com/streamich/react-use>

Напиши `usePrevious(value)`

ОПИСАНИЕ

Возвращает предыдущее значение prop / state. Полезно для compare.

КАК РАБОТАЕТ

- 01 `useRef`.
- 02 `useEffect`: `ref.current = value`.
- 03 `return ref.current` – старое значение до effect.
- 04 First render – `undefined`.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Animation triggers.
- 02 Diff detection.

ПОДВОДНЫЕ КАМНИ

- 01 Effect runs after render – есть subtle race.
- 02 First render – `undefined`.

МОГУТ ДОСПРОСИТЬ

- 01 Чем `useRef` от `useState` отличается?
- 02 Что такое effect timing?

ПОЛНАЯ СТАТЬЯ

<https://reactjs.org/docs/hooks-faq.html#how-to-get-the-previous-props-or-state>

ДЕНЬ 2

НАГРУЗКА

СРЕДНИЙ

3 ВОПРОСА

USE-FETCH

USE-ASYNC

USE-LOCALSTORAGE

ЧТО УЧИМ В ЭТОТ ДЕНЬ

USE-FETCH

USE-ASYNC

USE-LOCALSTORAGE

ВОПРОСЫ ДНЯ

Q04 Напиши `useFetch(url)`

Q05 Напиши `useAsync(asyncFn, deps)`

Q06 Напиши `useLocalStorage(key, initial)`

Напиши useFetch(url)

ОПИСАНИЕ

Hook возвращающий {data, error, loading}. AbortController на cleanup.

КАК РАБОТАЕТ

```
01  useState data, error, loading.  
02  useEffect(() => { const ctl = new AbortController(); fetch(url, {signal:  
    ctl.signal})... })  
03  return () => ctl.abort().  
04  deps [url].
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01  Quick fetch.  
02  Не для production – используй react-query.
```

ПОДВОДНЫЕ КАМНИ

```
01  Без AbortController – race condition + leak.  
02  Не handle cleanup race на быстрых url-changes.
```

МОГУТ ДОСПРОСИТЬ

```
01  Чем react-query от useFetch отличается?  
02  Что такое stale-while-revalidate?
```

ПОЛНАЯ СТАТЬЯ

<https://tanstack.com/query>

ВОПРОС Q05

Напиши useAsync(asyncFn, deps)

ОПИСАНИЕ

Generic для любых async-операций. Возвращает status + data + error.

КАК РАБОТАЕТ

- 01 useState status: 'idle' | 'pending' | 'success' | 'error'.
- 02 useEffect: setStatus pending, await fn, set success/error.
- 03 Cancel-flag для unmount.
- 04 Memoize asyncFn через useCallback.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Custom async logic.
- 02 Form submissions.

ПОДВОДНЫЕ КАМНИ

- 01 Stale closures для asyncFn.
- 02 Без cancel – setState после unmount.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое cancel-flag?
- 02 Чем useAsync от react-query отличается?

ПОЛНАЯ СТАТЬЯ

<https://github.com/streamich/react-use>

Напиши `useLocalStorage(key, initial)`

ОПИСАНИЕ

`useState` синхронизированный с `localStorage`. JSON `serialize` / `parse`.

КАК РАБОТАЕТ

```
01  useState(() => JSON.parse(localStorage.getItem(key)) ?? initial).
02  useEffect: localStorage.setItem(key, JSON.stringify(value)).
03  Listen на 'storage' event для cross-tab sync.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01  Persisted UI prefs.
02  Theme.
```

ПОДВОДНЫЕ КАМНИ

```
01  SSR – нет localStorage – нужен check.
02  JSON.parse throw – try-catch.
```

МОГУТ ДОСПРОСИТЬ

```
01  Что такое storage event?
02  Чем sessionStorage от localStorage отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://usehooks.com/uselocalstorage>

ДЕНЬ 3 НАГРУЗКА ТЯЖЁЛЫЙ 2 ВОПРОСА

AUTOCOMPLETE SEARCH WITH ABORT

ЧТО УЧИМ В ЭТОТ ДЕНЬ

AUTOCOMPLETE SEARCH WITH ABORT

ВОПРОСЫ ДНЯ

Q07 Напиши Autocomplete-компонент

Q08 Напиши search-input с AbortController

Напиши Autocomplete-компонент

ОПИСАНИЕ

Input + suggestions list. Debounce, AbortController, keyboard nav.

КАК РАБОТАЕТ

- 01 useDebounce input.
 - 02 useFetch suggestions.
 - 03 onKeyDown: ArrowUp/Down – focus.
 - 04 Enter – select.
 - 05 Esc – close.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Search.
 - 02 Mention picker.
 - 03 Tag picker.
-

ПОДВОДНЫЕ КАМНИ

- 01 Race condition без abort.
 - 02 Без debounce – слишком много запросов.
 - 03 Ally: aria-activedescendant.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое combobox pattern?
 - 02 Чем popover от tooltip отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/ARIA/apg/patterns/combobox/>

ВОПРОС Q08

Напиши search-input с AbortController

ОПИСАНИЕ

Каждый новый input cancel предыдущий fetch. useEffect cleanup.

КАК РАБОТАЕТ

```
01  useEffect(() => { const ctl = new AbortController(); fetch(url,
    {signal}).then(...) }).
02  return () => ctl.abort().
03  Catch AbortError – не показывать error.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01  Search.
02  Live filter.
03  Pagination.
```

ПОДВОДНЫЕ КАМНИ

```
01  abort выбрасывает Error – нужен check.
02  Без abort – race condition (старый ответ перетирает новый).
```

МОГУТ ДОСПРОСИТЬ

```
01  Что такое AbortError?
02  Чем AbortController от Promise.cancel отличается?
```

ПОЛНАЯ СТАТЬЯ

<https://developer.mozilla.org/en-US/docs/Web/API/AbortController>

INFINITE SCROLL

VIRTUAL LIST

ЧТО УЧИМ В ЭТОТ ДЕНЬ

INFINITE SCROLL VIRTUAL LIST

ВОПРОСЫ ДНЯ

Q09 Напиши Infinite Scroll

Q10 Напиши Virtual List (react-window-like)

ВОПРОС 009

Напиши Infinite Scroll

ОПИСАНИЕ

IntersectionObserver на sentinel. При intersect – fetch next page.

КАК РАБОТАЕТ

- 01 useRef для sentinel.
- 02 IntersectionObserver(callback, {root, rootMargin}).
- 03 onIntersect: setPage(p => p+1).
- 04 useEffect: observe + cleanup unobserve.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Feeds.
- 02 Search results.
- 03 Comments.

ПОДВОДНЫЕ КАМНИ

- 01 Без unobserve – leak.
- 02 Page race – добавление duplicates без id-dedup.
- 03 Loading false flicker.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое IntersectionObserver?
- 02 Чем infinite от pagination отличается?

ПОЛНАЯ СТАТЬЯ

https://developer.mozilla.org/en-US/docs/Web/API/Intersection_Observer_API

Напиши Virtual List (react-window-like)

ОПИСАНИЕ

Render только видимые items. Calculate offset через scrollTop / itemHeight.

КАК РАБОТАЕТ

- 01 useState scrollTop, useRef container.
 - 02 onScroll: setScrollTop.
 - 03 Visible: startIdx, endIdx из scrollTop / itemHeight.
 - 04 Render placeholder с total height + items с position: absolute / translateY.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Long lists.
 - 02 Tables.
 - 03 Chat messages.
-

ПОДВОДНЫЕ КАМНИ

- 01 Variable height – сложно.
 - 02 Без overscan – flicker на быстром scroll.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое overscan?
 - 02 Чем react-window от react-virtual отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://github.com/bvaughn/react-window>

ДЕНЬ 5 НАГРУЗКА ТЯЖЁЛЫЙ 4 ВОПРОСА

MODAL

PORTAL

FOCUS TRAP

OUTSIDE CLICK

ЧТО УЧИМ В ЭТОТ ДЕНЬ

MODAL PORTAL FOCUS TRAP OUTSIDE CLICK

ВОПРОСЫ ДНЯ

Q11 Напиши Modal с Portal

Q12 Напиши useFocusTrap для modal

Q13 Напиши useOnClickOutside(ref, handler)

Q14 Напиши Dropdown / Select с keyboard nav

Напиши Modal с Portal

ОПИСАНИЕ

Render в document.body через createPortal. Trap focus, prevent body scroll.

КАК РАБОТАЕТ

- 01 createPortal(<dialog>, document.body).
- 02 useEffect: document.body.style.overflow = 'hidden', cleanup.
- 03 Esc-handler.
- 04 Focus trap.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Confirm dialogs.
- 02 Forms.
- 03 Image preview.

ПОДВОДНЫЕ КАМНИ

- 01 Body scroll lock на iOS – нужен touchmove preventDefault.
- 02 Ally: role='dialog' + aria-modal.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое focus trap?
- 02 Чем Portal от inline render отличается?

ПОЛНАЯ СТАТЬЯ

<https://reactjs.org/docs/portals.html>

ВОПРОС Q12

Напиши useFocusTrap для modal

ОПИСАНИЕ

Focus остаётся внутри modal. Tab + Shift+Tab cycle через focusable elements.

КАК РАБОТАЕТ

- 01 useRef container.
- 02 useEffect: query 'a, button, input, [tabindex]'
- 03 onKeyDown Tab: если last → focus first; Shift+Tab + first → last.
- 04 Restore focus на closing.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Modals.
- 02 Drawers.
- 03 Menus.

ПОДВОДНЫЕ КАМНИ

- 01 Dynamic content – refresh focusable.
- 02 Без restore focus – UX-bug.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое :focus-visible?
- 02 Чем focus trap от inert отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/ARIA/apg/patterns/dialog-modal/>

Напиши `useOnClickOutside(ref, handler)`

ОПИСАНИЕ

Detect клик вне ref-элемента. Mousedown listener на document.

КАК РАБОТАЕТ

```
01 useEffect: document.addEventListener('mousedown', listener).
02 listener: if (ref.current && !ref.current.contains(target)) handler().
03 return () => removeEventListener.
04 mousedown лучше click – early.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Dropdowns.
02 Tooltips.
03 Menus.
```

ПОДВОДНЫЕ КАМНИ

```
01 click vs mousedown – разная UX.
02 Touch events отдельно (touchstart).
```

МОГУТ ДОСПРОСИТЬ

```
01 Чем mousedown от click отличается?
02 Что такое pointer events?
```

ПОЛНАЯ СТАТЬЯ

<https://usehooks.com/useonclickoutside>

ВОПРОС Q14

Напиши Dropdown / Select с keyboard nav

ОПИСАНИЕ

Click toggle. ArrowUp/Down – навигация. Enter – select. Esc – close.

КАК РАБОТАЕТ

- 01 useState open + activeIdx.
- 02 onKeyDown: switch по key.
- 03 onClick option: setValue, setOpen(false).
- 04 ARIA: role='listbox', aria-activedescendant.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Select.
- 02 Combobox.
- 03 Menu.

ПОДВОДНЫЕ КАМНИ

- 01 Без ally – screen reader broken.
- 02 Click + outside-close conflict.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое combobox vs listbox?
- 02 Чем native select от custom отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/ARIA/apg/patterns/listbox/>

FORM**VALIDATION****DROPDOWN****TABS****ACCORDION**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

FORM VALIDATION DROPDOWN TABS ACCORDION

ВОПРОСЫ ДНЯ

- Q15 Напиши контролируемую форму с валидацией
- Q16 Напиши Tabs-компонент с keyboard nav
- Q17 Напиши Accordion с single / multi mode
- Q18 Напиши Toast-нотификации
- Q19 Напиши Tooltip-компонент

ВОПРОС Q15

Напиши контролируемую форму с валидацией

ОПИСАНИЕ

useState для полей. onSubmit – validate, set errors, prevent default.

КАК РАБОТАЕТ

- 01 useState({email:'', password:''}).
- 02 onChange: setForm({...form, [name]: value}).
- 03 validate: return errors.
- 04 Show errors под полями.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Login.
- 02 Signup.
- 03 Settings.

ПОДВОДНЫЕ КАМНИ

- 01 State per field vs single object.
- 02 Без debounce – validate на каждый keystroke.

МОГУТ ДОСПРОСИТЬ

- 01 Чем react-hook-form лучше?
- 02 Что такое controlled component?

ПОЛНАЯ СТАТЬЯ

<https://react-hook-form.com/>

Напиши Tabs-компонент с keyboard nav

ОПИСАНИЕ

Click tab → switch. ArrowLeft/Right – navigate. ARIA tab pattern.

КАК РАБОТАЕТ

- 01 useState activeTab.
 - 02 Tabs map: button с role='tab', aria-selected.
 - 03 Panel с role='tabpanel'.
 - 04 onKeyDown: next/prev tab.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Settings page.
 - 02 Dashboard sections.
-

ПОДВОДНЫЕ КАМНИ

- 01 Без roving tabindex – broken focus.
 - 02 Не switch panel на arrow – нужен Enter/Space.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое roving tabindex?
 - 02 Чем Tabs от Accordion отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/ARIA/apg/patterns/tabs/>

ВОПРОС Q17

Напиши Accordion с single / multi mode

ОПИСАНИЕ

Click section → toggle expand. Single: один открыт, multi: несколько.

КАК РАБОТАЕТ

- 01 useState openIds: Set.
- 02 onClick: toggle id в Set.
- 03 ARIA: button с aria-expanded.
- 04 Анимация max-height.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 FAQ.
- 02 Settings groups.

ПОДВОДНЫЕ КАМНИ

- 01 max-height для unknown content – сложно.
- 02 Без animation – jarring UX.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое details/summary native?
- 02 Чем accordion от tabs отличается?

ПОЛНАЯ СТАТЬЯ

<https://www.w3.org/WAI/ARIA/apg/patterns/accordion/>

Напиши Toast-нотификации

ОПИСАНИЕ

Глобальный список toasts. Add → auto-dismiss через timeout. Portal.

КАК РАБОТАЕТ

- 01 Context + reducer.
- 02 addToast(msg, type).
- 03 setTimeout для auto-dismiss.
- 04 Portal для render над всем UI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Notifications.
- 02 Success / error feedback.

ПОДВОДНЫЕ КАМНИ

- 01 Без queue limit – overflow.
- 02 Ally: aria-live='polite'.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое aria-live?
- 02 Чем toast от alert отличается?

ПОЛНАЯ СТАТЬЯ

<https://sonner.emilkowal.ski/>

ВОПРОС Q19

Напиши Tooltip-компонент

ОПИСАНИЕ

Hover/focus → show. Mouseleave/blur → hide. Position via ref.

КАК РАБОТАЕТ

- 01 `useState visible.`
- 02 `onMouseEnter/onFocus: setVisible(true).`
- 03 `onMouseLeave/onBlur: false.`
- 04 Position через bounding rect или Floating UI.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Help text.
- 02 Icon labels.

ПОДВОДНЫЕ КАМНИ

- 01 Без focus – keyboard users не видят.
- 02 Hover-only – touch broken.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое Floating UI?
- 02 Чем tooltip от popover отличается?

ПОЛНАЯ СТАТЬЯ

<https://floating-ui.com/>

DRAG**SORT****TIC-TAC-TOE****UNDO****KANBAN**

ЧТО УЧИМ В ЭТОТ ДЕНЬ

DRAG SORT TIC-TAC-TOE UNDO KANBAN

ВОПРОСЫ ДНЯ

- Q20 Напиши Drag & Drop сортировку
- Q21 Напиши Tic-Tac-Toe
- Q22 Напиши Undo / Redo с history
- Q23 Напиши Calculator
- Q24 Напиши Color Picker (RGB sliders)
- Q25 Напиши простой Kanban (3 колонки + drag)

ВОПРОС Q20

Напиши Drag & Drop сортировку

ОПИСАНИЕ

Native HTML5 DnD или Pointer Events. onDragStart, onDragOver, onDrop.

КАК РАБОТАЕТ

```
01 draggable={true}.
02 onDragStart: e.dataTransfer.setData('id', id).
03 onDragOver: e.preventDefault.
04 onDrop: reorder array.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01 Sortable lists.
02 Kanban boards.
03 File upload.
```

ПОДВОДНЫЕ КАМНИ

```
01 Native DnD — ограничен.
02 react-dnd / dnd-kit лучше для сложных случаев.
```

МОГУТ ДОСПРОСИТЬ

```
01 Чем dnd-kit от react-dnd отличается?
02 Что такое pointer events?
```

ПОЛНАЯ СТАТЬЯ

<https://dndkit.com/>

Напиши Tic-Tac-Toe

ОПИСАНИЕ

3x3 grid. Click cell → place X/O. Determine winner. Reset.

КАК РАБОТАЕТ

```
01  useState board: Array(9).fill(null).
02  useState xIsNext.
03  onClick(i): if (winner || board[i]) return; setBoard.
04  calculateWinner: lines.
```

ГДЕ ВСТРЕЧАЕТСЯ

```
01  Algorithm interview.
02  Simple game.
```

ПОДВОДНЫЕ КАМНИ

```
01  Mutation board вместо нового array.
02  Не check для draw.
```

МОГУТ ДОСПРОСИТЬ

```
01  Чем minimax-алгоритм полезен?
02  Что такое immer для immutable?
```

ПОЛНАЯ СТАТЬЯ

<https://reactjs.org/tutorial/tutorial.html>

ВОПРОС Q 2 2

Напиши Undo / Redo с history

ОПИСАНИЕ

Stack past + present + future. Undo: pop past, push to future.

КАК РАБОТАЕТ

- 01 useReducer({past:[], present, future:[]}).
- 02 D0: future = [], past.push(present), present = new.
- 03 UNDO: future.unshift(present), present = past.pop().
- 04 REDO: обратное.

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Editor.
- 02 Canvas.
- 03 Forms.

ПОДВОДНЫЕ КАМНИ

- 01 Stacks без bound – memory leak.
- 02 Mutation вместо новых arrays.

МОГУТ ДОСПРОСИТЬ

- 01 Что такое immer.produce с patches?
- 02 Чем undo от history-API отличается?

ПОЛНАЯ СТАТЬЯ

<https://github.com/xolvio/use-undoable>

Напиши Calculator

ОПИСАНИЕ

Display + buttons. State: current, prev, op. Click – обновить.

КАК РАБОТАЕТ

- 01 useState {display, prev, op}.
 - 02 onDigit: append to display.
 - 03 onOp: store prev + op.
 - 04 onEquals: compute.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Algorithm interview.
 - 02 UI state machine.
-

ПОДВОДНЫЕ КАМНИ

- 01 Float arithmetic precision.
 - 02 Chain operations – нужна clear logic.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое state machine?
 - 02 Чем XState от useReducer отличается?
-

ПОЛНАЯ СТАТЬЯ

<https://xstate.js.org/>

ВОПРОС Q 2 4

Напиши Color Picker (RGB sliders)

ОПИСАНИЕ

3 input range RGB. Display preview. Output hex.

КАК РАБОТАЕТ

- 01 `useState {r, g, b}.`
 - 02 `Slider onChange: setRGB.`
 - 03 Preview через `style backgroundColor.`
 - 04 Compute hex: `('00' + r.toString(16)).slice(-2).`
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Theming.
 - 02 Customization.
-

ПОДВОДНЫЕ КАМНИ

- 01 Ally: label для slider.
 - 02 Без HSL – сложно подобрать цвет.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое HSL vs RGB?
 - 02 Чем react-colorful полезен?
-

ПОЛНАЯ СТАТЬЯ

<https://omgovich.github.io/react-colorful/>

Напиши простой Kanban (3 колонки + drag)

ОПИСАНИЕ

Колонки (TODO, IN-PROGRESS, DONE). Cards. Drag между колонками.

КАК РАБОТАЕТ

- 01 `useState` columns: {id, title, cards:[]}
 - 02 `onDragEnd`: move card from src column to dst.
 - 03 `dnd-kit` для DnD.
 - 04 Persist в `localStorage`.
-

ГДЕ ВСТРЕЧАЕТСЯ

- 01 Project boards.
 - 02 Task tracking.
-

ПОДВОДНЫЕ КАМНИ

- 01 Без unique id – drag broken.
 - 02 Reorder на drop – нужен правильный index.
-

МОГУТ ДОСПРОСИТЬ

- 01 Что такое `dnd-kit DragOverlay`?
 - 02 Чем `optimistic update` полезен?
-

ПОЛНАЯ СТАТЬЯ

<https://dndkit.com/>

ФИНАЛЬНЫЙ ЧЕК - ЛИСТ

СДАЛ ЛИ ЗАЧЁТ

ПРОЙДИ ВСЛУХ БЕЗ ПОДГЛЯДЫВАНИЙ ЕСЛИ ЗАСТРЯЛ – ВЕРНИСЬ В НУЖНЫЙ БИЛЕТ

- ☐ Написал `useDebounce` / `useThrottle` / `usePrevious`
- ☐ Написал `useFetch` с `AbortController` + `useAsync` + `useLocalStorage`
- ☐ Написал `Autocomplete` + `Search-input` с `abort`
- ☐ Написал `Infinite Scroll` (`IntersectionObserver`) + `Virtual List`
- ☐ Написал `Modal` + `Portal` + `focus trap` + `outside click`
- ☐ Написал `Dropdown` + `Tabs` + `Accordion` с `keyboard nav` + `ARIA`
- ☐ Написал `DnD` + `Sortable` + `Tic-Tac-Toe` + `Undo/Redo`
- ☐ Все компоненты `ally-friendly`

EVENT LOOP / ASYNC МЕТОДИЧКА v1

МАТЕРИАЛ ОПИРАЕТСЯ НА learn.javascript.ru